

WildKillLogger - Konfigurierbare Plugin-Dokumentation

Diese Dokumentation beschreibt die konfigurierbare Variante von **WildKillLogger** für **ARK: Survival Ascended**. Sie erklärt Architektur, Dateiausgaben, Heuristik, Konfiguration, Debugging, Erweiterbarkeit und enthält zusätzlich eine dokumentierte Referenz-Implementierung als Source-Datei.

1. Zweck und Grundidee

WildKillLogger erfasst PvE-Dino-Kills auf einem ASA-Dedicated-Server und schreibt verlässliche Kill-Datensätze in eine CSV-Datei. Da im getesteten Setup keine stabilen Damage-/Kill-Hooks wie `TakeDamage()` verfügbar waren, setzt das Plugin auf eine Nähe-Heuristik: `Dino-Destroyed()`-Events werden mit Spielerpositionen kombiniert.

Die Grundannahme ist: Wenn beim Verschwinden eines Dino-Actors genau **ein** frischer, naher Spieler gefunden wird, ist die Zuordnung als Kill ausreichend zuverlässig. Mehrdeutige Fälle werden nicht in die Produktiv-CSV geschrieben, sondern nur im Debug-Log dokumentiert.

2. Welche Dateien das Plugin erzeugt

Datei	Standardpfad	Bedeutung
config.json	ArkApi/Plugins/WildKillLogger/config.json	Laufzeitkonfiguration. Wird automatisch mit Default-Werten erzeugt.
wild_kills.csv	ArkApi/Plugins/WildKillLogger/wild_kills.csv	Produktive Datenquelle. Enthält nur verlässliche Kills (oder op...
wildkilllogger_debug.log	ArkApi/Plugins/WildKillLogger/wildkilllogger_debug.log	Diagnose-Log für Hook-Status, Tracking, Zuordnungen und v...

3. Datenquellen und worauf die Logik basiert

Das Plugin basiert auf drei Informationsquellen:

- 1) Join-Hook:** Über `AShooterGameMode.HandleNewPlayer_Implementation(...)` werden Spieler beim Join registriert. Gespeichert werden Controller, Character, Name, EOS-ID und erste Position.
- 2) Positions-Thread:** Ein Hintergrund-Thread aktualisiert alle `position_update_ms` Millisekunden die zuletzt bekannte Position jedes Spielers.
- 3) Destroyed-Hook:** Über `AActor.Destroyed()` werden Actor-Entfernungen erkannt. Nur Blueprint-Pfade, die wie echte Dino-Character-Blueprints aussehen, werden weiterverarbeitet.

Die Zuordnung eines Kills ist **heuristisch**, nicht physikalisch bewiesen. Das Plugin weiß nicht, wer Schaden verursacht hat, sondern nur, welche Spieler zum Zeitpunkt des Dino-Verschwindens frisch und in räumlicher Nähe waren.

4. Exakte Funktionsweise

Schritt A - Konfiguration laden: Beim Plugin-Start wird `config.json` geladen. Falls die Datei fehlt, wird eine Standarddatei erzeugt.

Schritt B - Hooks aktivieren: Das Plugin registriert den Join-Hook und den Destroyed-Hook.

Schritt C - Spieler tracken: Beim Join wird ein Eintrag in der internen Spielerliste angelegt. Der Positions-Thread aktualisiert Name, EOS-ID, Position und `last_seen`.

Schritt D - Dino-Filter: Bei jedem Destroyed-Event wird zuerst über Blueprint-Pfad gefiltert. Nicht-Dino-Objekte wie Buffs, `AIController`, `SupplyCrates` oder `DroppedItems` werden verworfen.

Schritt E - Attribution: Für Dino-Events wird die Dino-Position gelesen. Dann werden alle Spieler betrachtet, deren letzte Position frisch genug ist (Standard: höchstens 15 Sekunden alt) und deren Entfernung kleiner oder gleich dem Kill-Radius ist.

Schritt F - Confidence: Genau 1 Spieler im Radius = **high**. Mehrere Spieler = **low**. Kein Spieler = **unknown**.

Schritt G - CSV-Schreibregel: Standardmäßig werden nur **high**-Fälle in die CSV geschrieben. Alle anderen Fälle landen nur im Debug-Log.

5. Konfigurationsdatei

Die Datei `config.json` erlaubt die Laufzeitsteuerung ohne Rebuild.

```
{
  "kill_radius": 3000.0,
  "position_update_ms": 2000,
  "player_fresh_seconds": 15,
  "write_debug_log": true,
  "write_only_high_confidence": true,
  "debug_log_non_dino_destroy": false,
  "debug_log_skipped_kills": true,
  "kill_csv_filename": "wild_kills.csv",
  "debug_log_filename": "wildkilllogger_debug.log"
}
```

6. Bedeutung der Konfigurationsfelder

Feld	Typ	Default	Beschreibung
kill_radius	float	3000.0	Maximaler Abstand zwischen Dino und Spieler für eine mögliche Zuordnung.
position_update_ms	int	2000	Intervall des Hintergrund-Threads für Positionsupdates.
player_fresh_seconds	int	15	Spieler mit älteren Positionsdaten werden ignoriert.
write_debug_log	bool	true	Schreibt oder unterdrückt das Debug-Log.
write_only_high_confidence	bool	true	Wenn true, werden nur eindeutige Kills in die CSV geschrieben.
debug_log_non_dino_destroy	bool	false	Optionales Logging von verworfenen Nicht-Dino-Destroy-Events.
debug_log_skipped_kills	bool	true	Schreibt verworfene low/unknown-Fälle ins Debug-Log.
kill_csv_filename	string	wild_kills.csv	Name der CSV-Datei im Plugin-Ordner.
debug_log_filename	string	wildkilllogger_debug.log	Name des Debug-Logs im Plugin-Ordner.

7. Format von wild_kills.csv

```
timestamp_utc,dino_blueprint,dino_x,dino_y,dino_z,killer_eos,killer_name,nearest_distance
```

Diese Datei ist bewusst schlank gehalten, damit ein zweiter Bot oder ein externes Ranking-System sie leicht weiterverarbeiten kann. Mehrdeutige Fälle tauchen standardmäßig gar nicht auf.

8. Format und Zweck des Debug-Logs

Das Debug-Log dient als Diagnose- und Entwicklungswerkzeug. Typische Einträge:

```
[2026-03-22T09:00:00Z] PLUGIN_INIT
[2026-03-22T09:00:00Z] CONFIG kill_radius=3000.000000 position_update_ms=2000 ...
[2026-03-22T09:00:05Z] PLAYER_TRACKED eos_id=... name=Ollinator has_position=true
[2026-03-22T09:01:01Z] DINO_DESTROY blueprint=.../Para_Character_BP...
[2026-03-22T09:01:01Z] KILL_WRITTEN blueprint=.../Para_Character_BP... killer_name=Ollinator
```

Damit lassen sich Radius-Probleme, fehlende Spielerpositionen, Race Conditions oder unerwartete Destroyed-Ereignisse deutlich leichter nachvollziehen.

9. Warum die Header-Duplikate verschwunden sind

In der überarbeiteten Version wird der CSV-Header nur noch einmal geschrieben. Dafür gibt es die globale Variable `g_kill_csv_header_written` und einen geschützten Header-Write innerhalb eines File-Mutex. Dadurch werden doppelte Kopfzeilen bei parallelen Events verhindert.

10. Grundlegende Gedanken hinter dem Design

Stabilität vor Perfektion: Anstatt auf nicht verfügbare oder brüchige Combat-Hooks zu setzen, verwendet das Plugin nur Hooks, die im getesteten Setup tatsächlich funktionierten.

Saubere Daten statt vieler Daten: Es ist besser, nur eindeutige Kills zu schreiben, als viele potenziell falsche Zuordnungen zu produzieren.

Trennung von Produktiv- und Debug-Daten: CSV und Debug-Log sind bewusst getrennt. Die CSV bleibt sauber für Bots und Rankings, das Debug-Log bleibt ausführlich für Menschen.

Konfigurierbarkeit ohne Rebuild: Radius, Aktualisierungsintervall und Logging-Verhalten lassen sich per JSON-Datei anpassen, ohne die DLL neu bauen zu müssen.

11. Grenzen des Plugins

WildKillLogger beweist keine Schadensverursachung. Das Plugin sagt nicht: *Spieler X hat wirklich den letzten Schlag gemacht*. Es sagt: *Zum Zeitpunkt des Dino-Verschwindens war genau ein plausibler Spieler in Reichweite*.

Mehrere Spieler im selben Gebiet führen daher absichtlich zu **low** oder **unknown** und werden standardmäßig nicht in die CSV übernommen.

12. Inline dokumentierter Code - wichtigste Ausschnitte

Die vollständige dokumentierte Source-Datei liegt zusätzlich als Download bei. Hier die wichtigsten Kernstellen.

```
void LoadConfig()
{
    WriteDefaultConfigIfMissing();
    const std::string text = ReadTextFile(GetConfigPath());
    if (text.empty())
        return;

    Config cfg;
    cfg.kill_radius = RegexExtractFloat(text, "kill_radius", cfg.kill_radius);
    ...
    g_config = cfg;
}

void AttributeDinoDestroy(AActor* actor)
{
    const std::string blueprint = GetBlueprintPath(actor);
    if (!IsLikelyDinoCharacterBlueprint(blueprint))
        return;

    // Dino-Position bestimmen
    // frische Spieler im Radius suchen
    // genau 1 Spieler -> high confidence
    // nur dann in die CSV schreiben
}

if (should_write)
{
    AppendReliableKill(...);
    DebugLog("KILL_WRITTEN ...");
}
else if (g_config.debug_log_skipped_kills)
{
    DebugLog("KILL_SKIPPED ...");
}
```

13. Installation / Update

1. Die DLL in den Plugin-Ordner kopieren.
2. Die Datei `config.json` entweder vom Plugin erzeugen lassen oder selbst anlegen.
3. Server neu starten.
4. Im Plugin-Ordner prüfen, ob `wild_kills.csv` und `wildkilllogger_debug.log` entstehen.

14. Troubleshooting

Keine CSV-Einträge: Prüfen, ob nur low/unknown-Fälle auftreten. Das sieht man im Debug-Log.

Zu viele falsche Kills: Radius reduzieren.

Zu wenige Kills: Radius leicht erhöhen oder `player_fresh_seconds` prüfen.

Debug-Log zu groß: `write_debug_log` oder einzelne Debug-Flags reduzieren.

Viele Nicht-Dino-Einträge: `debug_log_non_dino_destroy` nur temporär einschalten.

15. Mitgelieferte Dateien

Zusätzlich zu dieser PDF wurden zwei Dateien erzeugt:

- `WildKillLogger_Main_config_documented.cpp` - dokumentierte Referenz-Implementierung
- `config.json.example` - Beispielkonfiguration